

a single .java file, if we write only one public class and multiple non-public classes  
then java compiler will generate how many .class files?

Whether the class is public OR non-public, Java compiler will generate the .class file for all the java classes.  
[Mantra: I have 10 java classes either public OR non-public so 10 .class files will be generated by compiler]  
Example :

-----  
//Test.java [File Name]

```
public class Test
{
}

class Alpha
{
}

class Beta
{
}
```

→ Total 3 .class files are created i.e. Test.class, Alpha.class and Beta.class

## Structure of a Java class?

- 1) package statement should be in the first line
- 2) import statement should be in the 2nd line
- 3) class body should be in the 3rd line

Example :

```
-----
package com.ravi.basic;

import java.util.Arrays;

public class Test
{
    Arrays a = null;
}
```

## Working with Static Method with different return type using BLC & ELC:

\* As we know we can directly call a static method with the help of class name, object is not required.

## Writing a java program with BLC & ELC class approach :

\* It is **highly recommended** to write java programs with BLC & ELC class approach. [We should never write everything (logic + execution) in a single program as shown below]

Main.java

```
-----
public class Main
{
    void main()
    {
        doSum(12,12);
    }

    public void doSum(int x, int y)
    {
        IO.println("Sum is :"+(x+y));
    }
}
```

//ELC

Main.java

```
-----
public class Main
{
    void main()
    {
        Addition.doSum(12,90);
    }
}
```

\* If we write the java programs with the help BLC & ELC approach then BLC classes can be re-usable into multiple packages providing loose coupling as shown below :

-----

//BLC

Addition.java

```
-----
public class Addition
{
    public static void doSum(int x, int y)
    {
        IO.println("Sum is :"+(x+y));
    }
}
```

//ELC

Main.java

```
-----
public class Main
{
    void main()
    {
        Addition.doSum(12,90);
    }
}
```

\* BLC & ELC PROGRAM STYLE APPROACH WE CAN WRITE BY USING FOLLOWING 2 APPROACHES :

1) Single File Approach :

-----

Main.java

-----

//ELC

public class Main

```
{
    void main()
    {
        Addition.doSum(12,90);
    }
}
```

-----

//BLC

class Addition

```
{
    public static void doSum(int x, int y)
    {
        IO.println("Addition of two number is :"+(x+y));
    }
}
```

-----

//ELC

public class Main

{

void main()

{

Addition.doSum(10,90);

}

}

-----

//BLC

class Addition

{

public static void doSum(int x, int y)

{

IO.println("Sum is :"+(x+y));

}

}

-----

//Programs :

-----

Circle.java

-----

public class Circle //public

{

}

}

-----

class Square //non public

{

}

}

-----

b/c

(folder)

Main.java

-----

public class Main

{

void main()

{

Valid I can use

circle class here

}

}

-----

elc

(folder)

\* If we want to use the BLC class from one **package to another package** then we need to import the BLC class from current package (folder) to another package (folder) by using **import statement** but the BLC class **must be public**

\* **If BLC class is non public then we cannot import to another package (folder)**

Circle.java

-----

package com.ravi.b/c;

-----

public class Circle //public

{

public static void getAreaOfCircle(double radius)

{

final double PI = 3.14;

double area = PI \* radius \* radius;

System.out.printf("Area of Circle is : %.2f ",area);

}

}

-----

package com.ravi.elc;

-----

import com.ravi.b/c.Circle;

-----

public class Main

{

void main()

{

double radius = Double.parseDouble(IO.readLine("Enter the radius :"));

Circle.getAreaOfCircle(radius);

}

}

-----

The above program is available with multiple file approach

-----

package com.ravi.b/c;

-----

//BLC

public class ArithmeticOperation

{

public static int sum(int x, int y)

{

return x+y;

}

-----

public static int sub(int x, int y)

{

return x-y;

}

-----

public static int mul(int x, int y)

{

return x\*y;

}

-----

public static int div(int x, int y)

{

if(y==0)

{

System.err.println("Cannot divide a number by zero");

return 0;

}

-----

return x/y;

}

}

-----

package com.ravi.elc;

-----

import com.ravi.b/c.ArithmeticOperation;

-----

public class Main

{

void main()

{

int result = ArithmeticOperation.sum(12, 12);

IO.println("Sum is :"+result);

-----

IO.println("Sub is :"+ArithmeticOperation.sub(200, 100));

IO.println("Mul is :"+ArithmeticOperation.mul(10, 20));

IO.println("Div is :"+ArithmeticOperation.div(200, 10));

}

}

-----

Working with method return type as a String :

-----

\* If a method return type is String then we can append multiple values in a single statement by using concatenation operator.

package com.ravi.b/c;

-----

public class Employee

{

public static String getEmployeeDetails(int id, String name, double salary)

{

return "[Employee id is :"+id+", name is :"+name+", salary is :"+salary+"]";

}

}

-----

package com.ravi.elc;

-----

import com.ravi.b/c.Employee;

-----

public class Main

{

void main(String [] args)

{

int id = Integer.parseInt(args[0]);

String name = args[1];

double salary = Double.parseDouble(args[2]);

-----

String employeeDetails = Employee.getEmployeeDetails(id, name, salary);

IO.println("Employee Details :");

IO.println(employeeDetails);

}

}

-----

How to execute command line argument program in Eclipse IDE :

Right click on the program -> Run as -> Run Configuration -> Verify your project name & Main class name (ELC class) -> Provide appropriate value to the argument tab -> run the program